GraphQL

🔥 **Hot Tips**

- Auto-generate a full GraphQL API from your ORM classes with a single `schema.from_orm(User)` call
- Zero external dependencies — Tina4 Ruby ships its own recursive-descent parser and executor
- Register the `/graphql` endpoint in one line
- Supports queries, mutations, variables, fragments, and aliases out of the box

Quick Start

The fastest way to get a working GraphQL API is to point it at an existing ORM class.

```ruby
require "tina4"

gql = Tina4::GraphQL.new
gql.schema.from_orm(User)

gql.register_route("/graphql")

Tina4.run_web_server
```

That single `from_orm` call creates:

Generated	Name	Description
Type	`User`	Object type with fields from ORM field definitions
Query	`user(id: ID)`	Fetch a single record by ID
Query	`users(limit: Int, offset: Int)`	Paginated list (default limit 10)
Mutation	`createUser(input: UserInput)`	Insert a new record
Mutation	`updateUser(id: ID!, input: UserInput)`	Update an existing record
Mutation	`deleteUser(id: ID!)`	Delete a record, returns Boolean

You can now POST to `/graphql` and start querying. A GET request to `/graphql` serves the GraphiQL interactive IDE.

Setting Up the Endpoint

`register_route` creates both a `POST /graphql` handler and a `GET /graphql` GraphiQL UI:

ruby

```ruby
gql = Tina4::GraphQL.new

# Default path
gql.register_route("/graphql")

# Custom path
gql.register_route("/api/graphql")
```

If you need more control (authentication, custom context, etc.), wire the route yourself:

ruby

```ruby
gql = Tina4::GraphQL.new

Tina4.post "/graphql", auth: false do |request, response|
```

```ruby
  body = request.json_body
  result = gql.execute(body["query"], body["variables"] || {})
  response.json(result)
end
```

Defining a Schema Manually

When auto-generation does not fit your needs, build the schema by hand.

Register a type

ruby

```ruby
schema = Tina4::GraphQLSchema.new

# Define an OBJECT type with named fields
schema.add_type("Product", {
  "id"      => "ID",
  "name"    => "String",
  "price"   => "Float",
  "inStock" => "Boolean"
})
```

Register a query

ruby

```ruby
schema.add_query("product", {
  type: "Product",
  args: { "id" => "ID" },
  resolve: ->(root, args, context) {
    { "id" => args["id"], "name" => "Widget", "price" => 9.99, "inStock" => tru
  }
})
```

Register a mutation

ruby

```ruby
schema.add_mutation("updateProductPrice", {
  type: "Product",
  args: { "id" => "ID!", "price" => "Float!" },
  resolve: ->(root, args, context) {
    # your update logic here
    { "id" => args["id"], "name" => "Widget", "price" => args["price"], "inStoc
  }
})
```

Auto-Generating Schema from ORM

Given a Tina4 ORM class, `from_orm` introspects its field definitions, maps Ruby types to GraphQL scalars, and registers CRUD queries and mutations automatically.

How it works

1. ORM field definitions (e.g. `string_field :name`) become fields on the GraphQL type.
2. Field types are mapped: `integer_field` → `Int` , `numeric_field` → `Float` , `string_field` / `text_field` → `String` .
3. The primary key field is mapped to `ID` .
4. The class name is used as the type name. Lower-case is used for query names.

Example ORM class

ruby

```ruby
class Product < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field  :name
  numeric_field :price
  integer_field :category_id
end
```

Register it

ruby

```ruby
schema = Tina4::GraphQLSchema.new
schema.from_orm(Product)
schema.from_orm(Category)

gql = Tina4::GraphQL.new(schema)
gql.register_route("/graphql")
```

You can call `from_orm` multiple times to register as many ORM classes as you like on the same schema.

Type mapping reference

ORM Field Type	GraphQL Type
`integer_field`	`Int`
`numeric_field`	`Float`
`string_field`	`String`
`text_field`	`String`
`datetime_field`	`String`
`blob_field`	`String`
Primary key field	`ID`

Writing Queries

Send a JSON POST request to your endpoint:

json

```json
{
  "query": "{ users(limit: 5) { id name email } }"
}
```

Named query

graphql

```graphql
query GetProduct($productId: ID) {
  product(id: $productId) {
    id
    name
    price
  }
}
```

With variables:

json

```json
{
  "query": "query GetProduct($productId: ID) { product(id: $productId) { id nam
  "variables": { "productId": "42" }
}
```

Aliases

Request the same field multiple times with different arguments using aliases:

graphql

```graphql
{
  first: product(id: "1") { name price }
  second: product(id: "2") { name price }
}
```

Writing Mutations

Mutations follow the same POST format. Use the `mutation` keyword:

graphql

```graphql
mutation {
  createProduct(input: { name: "Gadget", price: 19.99, categoryId: 3 }) {
    id
```

```
    name
  }
}
```

Update:

graphql

```graphql
mutation {
  updateProduct(id: "42", input: { price: 24.99 }) {
    id
    price
  }
}
```

Delete:

graphql

```graphql
mutation {
  deleteProduct(id: "42")
}
```

Resolvers

Every query and mutation field needs a `resolve` callable. The signature is:

ruby

```ruby
->(root, args, context) { ... }
```

Parameter	Description
`root`	The parent value. `nil` for root-level fields.
`args`	Arguments passed in the query, with variables already resolved.
`context`	Whatever you passed as context when calling `execute()` .

For nested fields, the executor resolves automatically by reading hash keys from the parent value. You only need explicit resolvers for root-level queries and mutations.

ruby

```ruby
schema.add_query("topProducts", {
  type: "[Product]",
  args: { "limit" => "Int" },
  resolve: ->(root, args, context) {
    limit = args["limit"] || 5
    Product.select(limit: limit).records
  }
})
```

Variables and Fragments

Variables

Declare variables in the operation definition and pass their values in the `variables` JSON field:

graphql

```graphql
query ListProducts($limit: Int, $offset: Int) {
  products(limit: $limit, offset: $offset) {
    id
    name
  }
}
```

json

```json
{
  "query": "query ListProducts($limit: Int, $offset: Int) { products(limit: $li
  "variables": { "limit": 10, "offset": 20 }
}
```

Fragments

Fragments let you reuse field selections across queries:

graphql

```graphql
fragment ProductFields on Product {
  id
  name
  price
}

query {
  product(id: "1") {
    ...ProductFields
  }
  products(limit: 3) {
    ...ProductFields
  }
}
```

GraphiQL IDE

A GET request to the GraphQL endpoint serves the GraphiQL interactive IDE, giving you a browser-based query editor with auto-complete and documentation.

```
http://localhost:7145/graphql
```

Programmatic Usage

You do not have to use the HTTP endpoint. The `GraphQL` class can be called directly in your code:

ruby

```ruby
gql = Tina4::GraphQL.new
gql.schema.from_orm(User)
```

```ruby
result = gql.execute('{ users(limit: 3) { id name } }')
# => { "data" => { "users" => [...] } }
```

Pass variables as the second argument:

```ruby
result = gql.execute(
  'query ($id: ID) { user(id: $id) { id name } }',
  { "id" => "42" }
)
```

Error Handling

Errors are returned in the standard GraphQL `errors` array alongside `data`:

```json
{
  "data": { "user": null },
  "errors": [
    { "message": "Field 'user' not found in query", "path": ["user"] }
  ]
}
```

If the query itself fails to parse, the response will have `data: null` and the parser error in `errors`.

Full Working Example

```ruby
require "tina4"

# ORM classes
class Category < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field  :name
```

```ruby
end

class Product < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field  :name
  numeric_field :price
  integer_field :category_id
end

# Initialize database
Tina4::ORM.init(Tina4::Database.new("sqlite3:shop.db"))

# Build GraphQL schema
gql = Tina4::GraphQL.new
gql.schema.from_orm(Category)
gql.schema.from_orm(Product)

# Add a custom query alongside the auto-generated ones
gql.schema.add_query("cheapProducts", {
  type: "[Product]",
  args: { "maxPrice" => "Float" },
  resolve: ->(root, args, context) {
    max = args["maxPrice"] || 10.0
    Product.select(filter: "price <= ?", params: [max]).records
  }
})

gql.register_route("/graphql")

Tina4.run_web_server
```

Now you can query:

graphql

```graphql
{
  # Auto-generated
  categories(limit: 100) { id name }

  # Auto-generated
  product(id: "7") { id name price }
```

```
  # Custom
  cheapProducts(maxPrice: 5.00) { id name price }
}
```

Further Reading

- ORM — the ORM classes that power `from_orm`
- Basic Routing — how Tina4 routing works under the hood
- REST API — for traditional REST-style endpoints

Previous page
Development Mode

Next page
Queues